Generalizability (LM)

Natural language processing

Ayesha Enayet

- Domain adaptation
- Domain shift
- Overfitting
- Probabilities often encode specific facts about a given training corpus

Domain adaptation

- Basically a machine learning concept used to improve the performance of a model trained on data from one domain (the **source domain**) when it is applied to a different but related domain (the **target domain**).
- Can be applied to **N-gram models as well.**

- N-gram models rely on the statistical co-occurrence of words in a corpus. When trained on data from one domain (e.g., news articles), the N-grams and their probabilities may not generalize well to a different domain (e.g., social media posts), where word usage, vocabulary, or syntax might vary. Domain adaptation techniques can be used to improve the performance of N-gram models when they are applied to a different domain.
- How?

Shakespeare vs WSJ corpus

1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

Figure 3.3 Eight sentences randomly generated from four n-grams computed from Shakespeare's works. All characters were mapped to lower-case and punctuation marks were treated as words. Output is hand-corrected for capitalization to improve readability.

1 gram	Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives
2 gram	Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
3 gram	They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Figure 3.4 Three sentences randomly generated from three n-gram models computed from 40 million words of the *Wall Street Journal*, lower-casing all characters and treating punctuation as words. Output was then hand-corrected for capitalization to improve readability.

Statistical model (limitation)

- Statistical models are likely to be pretty useless as predictors if the training sets and the test sets are as different as Shakespeare and WSJ.
- Solution:
  - Choose training set wisely

Unknown word

- Out of vocabulary (OOV) word
- There are two common ways to train the probabilities of the unknown word model . **The first** one is by choosing a fixed vocabulary in advance:

1. Choose a vocabulary (word list) that is fixed in advance.
2. Convert in the training set any word that is not in this set (any OOV word) to the unknown word token in a text normalization step.
3. Estimate the probabilities for from its counts just like any other regular word in the training set.

Second alternative

- In situations where we don’t have a prior vocabulary in advance, is to create such a vocabulary implicitly, replacing words in the training data by based on their frequency. For example we can replace by all words that occur fewer than n times in the training set, where n is some small number, or equivalently select a vocabulary size V in advance (say 50,000) and choose the top V words by frequency and replace the rest by UNK. In either case we then proceed to train the language model as before, treating like a regular word.

Zero probability n-gram

- Zero probability n-gram
  - their presence means we are underestimating the probability of all sorts of words that might occur, which will hurt the performance of any application we want to run on this data.
  - Second, if the probability of any word in the test set is 0, the entire probability of the test set is 0.
- Can we compute perplexity?
  - By definition, perplexity is based on the inverse probability of the test set. Thus if some words have zero probability, we can’t compute perplexity at all, since we can’t divide by 0!

- A key problem in N-gram modeling is the inherent data sparseness.
- For example, in several million words of English text, more than 50% of the trigrams occur only once; 80% of the trigrams occur less than five times.
- Higher order N-gram models tend to be domain or application specific. Smoothing provides a way of generating generalized language models.
- If an N-gram is never observed in the training data, can it occur in the evaluation data set?

https://isip.piconepress.com/courses/msstate/ece_8463/lectures/current/lecture_33/lecture_33.pdf

Smoothing

- What do we do with words that are in our vocabulary (they are not unknown words) but appear in a test set in an unseen context (for example they appear after a word they never appeared after in training)?

- Solution: Smoothing is the process of flattening a probability distribution implied by a language model so that all reasonable word sequences can occur with some probability. This often involves broadening the distribution by redistributing weight from high probability regions to zero probability regions.

Laplace smoothing (add-1 smoothing)

- Pretend each bigram occurs once more than it actually does in the training data set.
- All the counts that used to be zero will now have a count of 1, the counts of 1 will be 2, and so on.

$$P(w_i) = \frac{c_i}{N}$$

$$P_{\text{Laplace}}(w_i) = \frac{c_i + 1}{N + V}$$

Bigram (Add-1 smoothing)

- $P(w_2|w_1)=\frac{count(w_1,w_2)+1}{count(w_1)+V}$

Change in bigram counts

	i	want	to
i	5	827	0
want	2	0	608
to	2	0	4
eat	0	0	2
chinese	1	0	0

	i	want	to
i	6	828	1
want	3	1	609
to	3	1	5
eat	1	1	3
chinese	2	1	1

Discounting

- Discount d is a ratio between the new and the old counts.
- The counts for each prefix word have been reduced; the discount for the bigram ***want to*** is .39

	i	**want**	**to**
i	0.002	0.33	0
want	0.0022	0	0.66
to	0.00083	0	0.0017
eat	0	0	0.0027
chinese	0.0063	0	0

	i	**want**	**to**
i	0.0015	0.21	0.00025
want	0.0013	0.00042	0.26
to	0.00078	0.00026	0.0013
eat	0.00046	0.00046	0.0014
chinese	0.0012	0.00062	0.00062

Drawbacks:

- **Over-smoothing**: Since Laplace smoothing adds a constant to all N-grams, it tends to assign too much probability mass to rare or unseen events, which can negatively impact performance, especially when the vocabulary is large.

Impact on Large Vocabularies:

- In a large vocabulary, adding 1 to every possible event can cause significant problems:
- **Infrequent N-grams:** Since the vocabulary size V is large, the amount subtracted from frequent events is large.
- **Impact on Model Performance:** The model's performance can deteriorate because rare or unseen events dominate the probabilities.
- **Effect:** The model's generalization ability suffers because it does not effectively differentiate between very frequent and rare/unseen events. This is particularly problematic in tasks like **language modeling**, where many words and word combinations are not observed in training data.

Add-K smoothing

- Instead of adding 1 to each count, we add a fractional count k (.5? .05? .01?)
- **Choosing k**: The main challenge is selecting the optimal value of k. If k is too large, it may still overestimate unseen events, and if too small, it may not sufficiently smooth the model.

$$P^*_{\text{Add-k}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+k}{C(w_{n-1})+kV}$$

- How to choose k?

- Split your dataset into training and validation sets (e.g., 80/20 split).
- Train your model on the training set using various values of k (e.g., $k \in \{0.01, 0.1, 0.5, 1, 2\}$.
- Evaluate the model's performance (e.g., using perplexity or accuracy) on the validation set.
- Select the k that minimizes the validation error.
- Disadvantage?

Backoff

- Is for N-gram models
- If we are trying to compute $P(w_n|w_{n-2}w_{n-1})$ but we have no examples of a particular trigram $w_{n-2}w_{n-1}w_n$, we can instead estimate its probability by using the bigram probability $P(w_n|w_{n-1})$. Similarly, if we don't have counts to compute $P(w_n|w_{n-1})$, we can look to the unigram $P(w_n)$.
- In backoff, we use the trigram if the evidence is sufficient, otherwise we use the bigram, otherwise the unigram. In other words, we only "back off" to a lower-order n-gram if we have zero evidence for a higher-order interpolation n-gram.

Interpolation

- In interpolation, we always mix the probability estimates from all the n-gram estimators, weighing and combining the trigram, bigram, and unigram counts.

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ &\quad +\lambda_2 P(w_n|w_{n-1}) \\ &\quad +\lambda_3 P(w_n)\end{aligned}$$

such that the λs sum to 1:

$$\sum_i \lambda_i = 1$$

Finding λ ☺

- A held-out corpus is an additional training corpus that we use to set hyperparameters like these λ values, by choosing the λ values that maximize the likelihood of the held-out corpus.

- That is, we fix the n-gram probabilities and then search for the λ values that—when plugged into the equation—give us the highest probability of the held-out set.